
UNIVERSIDADE FEDERAL DO AMAZONAS

Programa de Pós-Graduação em Engenharia Elétrica — PPGE

Linha 1 — Sistemas Inteligentes e Microeletrônica

Lista de Exercícios 02

Verificação de Sistemas Digitais e k -Indução

PGENE601/PPGINF554 — Verificação e Síntese Automática
de Sistemas Ciber-Físicos e Embarcados

Semana 5 (28/09/2026 a 04/10/2026)

Disciplina: Verificação e Síntese Automática de Sist. Ciber-Físicos e Emb.

Docente: Prof. Dr. Lucas C. Cordeiro

Discente: Matheus Serrão Uchôa

Nível: Pós-Graduação — Mestrado

Artigos-base: DSVerifier (SPIN 2015) e k -Indução (STTT 2017)

Data: Manaus, 16 de setembro de 2026

Sumário

Artigos de Referência	2
1 Efeitos de <i>finite word length</i> (Artigo A)	3
1.1 (a) Faixa representável e resolução do formato $\langle 4, 10 \rangle$	3
1.2 (b) Os coeficientes e a faixa $[-5, 5]$ cabem em $\langle 4, 10 \rangle$?	3
1.3 (c) Por que a inicialização precisa ocorrer antes da instrumentação	4
1.4 (d) Oscilação persistente com entrada constante	5
2 Arquitetura e completude do DSVerifier (Artigo A)	6
2.1 (a) Fluxo de verificação e posição do módulo	6
2.2 (b) ANSI-C em vez de SMT direto: uma vantagem e uma limitação	6
2.3 (c) A assimetria entre estabilidade/fase mínima e as demais propriedades	7
2.4 (d) Aumentar <code>--x-size</code> não estabelece corretude	7
3 Os três casos da k-indução (Artigo B)	9
3.1 (a) Por que BMC tradicional falha e quantos desdobramentos seriam necessários	9
3.2 (b) Transformação do caso base para $k = 2$	10
3.3 (c) Transformação da condição de avanço para $k = 2$	10
3.4 (d) Por que a ordem base $\rightarrow$ avanço/indutivo importa	11
4 O passo indutivo e suas transformações (Artigo B)	13
4.1 (a) Função de cada bloco	13
4.2 (b) O bloco R e a terminação do passo indutivo	13
4.3 (c) Havoc apenas das variáveis do laço vs. Große et al. (todas as variáveis)	14
4.4 (d) Por que a hipótese $\{Post \wedge \neg(i \leq n)\}$ prova o caso geral	15
5 Integração entre os dois artigos	16
5.1 (a) Qual dos três casos fecha a lacuna de solidez do Artigo A	16
5.2 (b) Alterações necessárias no fluxo do DSVerifier	16
5.3 (c) O código gerado pelo DSVerifier respeita as restrições do passo indutivo?	17
5.4 (d) Mais resultados corretos e mais <i>wrong proofs</i> : o que pesa mais?	18

Artigos de Referência

Artigo A. H.I. Ismail, I.V. Bessa, L.C. Cordeiro, E.B. de Lima Filho e J.E. Chaves Filho. *DSVerifier: A Bounded Model Checking Tool for Digital Systems*. SPIN 2015, LNCS 9232, pp. 126–131.

Artigo B. M.Y.R. Gadelha, H.I. Ismail e L.C. Cordeiro. *Handling Loops in Bounded Model Checking of C Programs via k -Induction*. STTT, 2017.

Todas as respostas abaixo citam diretamente o texto dos artigos (caixas cinzas) sempre que a justificativa depende da formulação exata usada pelos autores.

Exercício 1. Efeitos de *finite word length* (Artigo A)

Enunciado

Controlador de segunda ordem $H(z) = \frac{2,813z^2 - 0,0163z - 1,872}{z^2 + 1,068z + 0,1239}$, planta de motor de indução, período de amostragem 0,5s, formato `int_bits=4`, `frac_bits=10`, `min=-5.0`, `max=5.0`.

1.1 (a) Faixa representável e resolução do formato $\langle 4, 10 \rangle$

O Artigo A usa a convenção de ponto fixo em que `int_bits` *inclui* o bit de sinal (a struct `implementation` declara diretamente `int_bits` e `frac_bits`, sem um campo separado de sinal). Em complemento de dois, com $I = 4$ bits inteiros (sinal incluso) e $F = 10$ bits fracionários, a palavra tem $I + F = 14$ bits e representa valores $v = m \cdot 2^{-F}$, onde m é um inteiro em complemento de dois de $I + F$ bits:

$$m \in [-2^{I+F-1}, 2^{I+F-1} - 1] = [-8192, 8191].$$

$$v_{\min} = -2^{I-1} = -2^3 = -8,000000$$

$$v_{\max} = 2^{I-1} - 2^{-F} = 8 - 2^{-10} = 8 - 0,0009765625 = 7,9990234375$$

$$\Delta = 2^{-F} = 2^{-10} = 0,0009765625 \approx 9,7656 \times 10^{-4}$$

Resultado

Faixa representável $\langle 4, 10 \rangle$: $[-8,0; 7,999023]$. Resolução (LSB): $\Delta = 2^{-10} \approx 0,000977$.

1.2 (b) Os coeficientes e a faixa $[-5, 5]$ cabem em $\langle 4, 10 \rangle$?

Os coeficientes de $H(z)$, conforme o próprio arquivo de especificação do DSVerifier (`ds.b` = {2.813, -0.0163, -1.872}, `ds.a` = {1.0, 1.068, 0.1239}), e a faixa de entrada $[-5,0; 5,0]$:

Valor	Dentro de $[-8,0; 7,999023]$?	
$b_0 = 2,813$	Sim	$-8 \leq 2,813 \leq 7,999$
$b_1 = -0,0163$	Sim	
$b_2 = -1,872$	Sim	
$a_0 = 1,0$	Sim	
$a_1 = 1,068$	Sim	
$a_2 = 0,1239$	Sim	
$[-5,0; 5,0]$	Sim	$[-5, 5] \subset [-8; 7,999]$

Resultado

Tudo cabe. Nenhum coeficiente nem a faixa de entrada excede $[-8,0; 7,999023]$, logo $\langle 4, 10 \rangle$ é um formato válido em termos de *faixa* (não há overflow estrutural nos valores nominais). Isso não implica representação **exata**: por exemplo 0,1239 não é um múltiplo de 2^{-10} e será arredondado para o múltiplo mais próximo, $130/1024 \approx 0,126953$ (erro de quantização $\approx 3 \times 10^{-3}$) — esse é precisamente o efeito de *finite word length* que o Artigo A se propõe a verificar (overflow *durante a execução* do filtro, quando somas/produtos intermediários de estados internos, não apenas os coeficientes nominais, podem extrapolar a faixa).

1.3 (c) Por que a inicialização precisa ocorrer antes da instrumentação**Citação direta**

“DSVerifier performs three main procedures: initialization, validation, and instrumentation [...] the first step is to initialize its internal parameters for quantization, that is, it computes the maximum and minimum representable numbers for the chosen FWL format. [...] In the last step, DSVerifier adds explicit calls to the verification engine (for the evaluated properties), using functions available in ESBMC (e.g., `__ESBMC_assume` and `__ESBMC_assert`), in order to check for property violations.” (Artigo A, Seção 2)

A instrumentação é o passo que **escreve, como texto C literal**, as chamadas `__ESBMC_assert(x <= max && x >= min)` (e equivalentes para overflow/limit cycle) dentro do arquivo ANSI-C gerado. Para emitir essas linhas, a instrumentação precisa já conhecer os valores numéricos concretos de **max** e **min** — eles entram como **constantes literais** no código gerado, não como expressões a resolver depois.

Nota didática

O ESBMC, que roda **depois** (Fig. 1 do Artigo A: C Parser $\rightarrow$ GOTO Program $\rightarrow$ GOTO Symex $\rightarrow$ SMT Solver), é um *model checker de C genérico*: ele não tem nenhuma noção embutida de “ponto fixo”, “FWL” ou “formato $\langle k, l \rangle$ ” — só enxerga `assert/assume` com constantes literais. Se o cálculo de `max/min` fosse adiado para “durante a verificação”, não haveria mais nenhum componente no pipeline capaz de calculá-lo: o DSVerifier já delegou o controle ao ESBMC nesse ponto, e o arquivo ANSI-C já foi escrito e fechado. Por isso a inicialização é necessariamente uma etapa de *geração de código* (tempo de compilação/instrumentação), estritamente anterior e externa à verificação propriamente dita.

1.4 (d) Oscilação persistente com entrada constante**Citação direta**

“Limit Cycle. There can be persistent oscillations in the output of a system with constant input. DSVerifier is able to check for zero-input limit cycles, for any initial condition.” (Artigo A, Seção 2.1)

Resultado

A propriedade é **Limit Cycle** (ciclo-limite), definida no artigo exatamente como oscilação persistente na saída de um sistema com entrada constante (o artigo verifica, em particular, ciclos-limite de entrada nula — *zero-input* — para qualquer condição inicial).

Exercício 2. Arquitetura e completude do DSVerifier (Artigo A)

2.1 (a) Fluxo de verificação e posição do módulo

Citação direta

“DSVerifier is an internal module for the efficient SMT-based context-bounded model checker (ESBMC) [...]. The complete verification tool includes four components from ESBMC [...]: C parser, GOTO Program, GOTO Symex, and SMT solver. [...] The DSVerifier module is included *before* the C parser.” (Artigo A, Seção 2, Fig. 1)

Fluxo completo: *Digital System Specification* → **DSVerifier** (internamente: Initialization → Validation → Instrumentation) → arquivo ANSI-C → **C Parser** → **GOTO Program** → **GOTO Symex** → **SMT Solver** (Boolector).

Nota didática

É a única posição possível porque o que o módulo **produz** é, literalmente, um arquivo-texto ANSI-C (“an ANSI-C code file is generated [...] this file is directly sent to the C parser module”). O próximo estágio capaz de consumir texto C é, por definição, o C Parser — o *front-end* de qualquer *toolchain* baseada em fonte C. Não pode entrar depois do C Parser (GOTO Program já espera uma representação intermediária, não texto-fonte), nem depois do GOTO Symex ou no SMT Solver (o DSVerifier não tem nenhuma noção de IR-GOTO ou de fórmula SMT — ele só sabe emitir C). A posição é ditada inteiramente pelo tipo do artefato de saída.

2.2 (b) ANSI-C em vez de SMT direto: uma vantagem e uma limitação

Vantagem. O DSVerifier reaproveita integralmente o *pipeline* maduro do ESBMC (parser, tradução GOTO, execução simbólica, codificação SMT e múltiplos *back-ends* de solver intercambiáveis) sem precisar implementar seu próprio codificador SMT ou semântica de *bit-vectors*; basta saber gerar C corretamente. Como o próprio artigo nota, o arquivo gerado “can be verified by any C model-checker that supports bit-vector reasoning” — portabilidade/independência de ferramenta.

Limitação. A qualidade da verificação fica inteiramente refém da semântica genérica de C que o ESBMC oferece: o DSVerifier não pode expressar nem explorar nenhuma otimização específica de domínio (por exemplo, um procedimento de decisão dedicado a aritmética de ponto fixo Q-format), pois tudo precisa antes ser reduzido a operações inteiras/*bit-vector* de C puro; qualquer imprecisão, inconsistência de solidez (como a in-

completude discutida no Artigo B) ou escolha de modelo de memória do ESBMC é herdada de forma transparente, sem que o DSVerifier tenha qualquer oportunidade de intervir no nível da fórmula SMT.

2.3 (c) A assimetria entre estabilidade/fase mínima e as demais propriedades

Citação direta

“The verification of stability and minimum-phase is complete and sound, since it does not depend on system outputs and inputs. However, the verification of other property types is typically unsound, unless some induction technique is used.” (Artigo A, Seção 2.2)

Estabilidade e fase mínima são propriedades **puramente algébricas dos coeficientes** do filtro (localização de pólos/zeros no plano z): dado um conjunto **fixo e finito** de coeficientes quantizados, verificar se todos os pólos (ou zeros) estão dentro do círculo unitário é um cálculo fechado, de tamanho constante, que **dispensa completamente o raciocínio sobre execuções** — não é preciso desenrolar nenhum laço de controle nem simular a operação do filtro ao longo do tempo; não há “ k ” a escolher, porque não há sequência temporal de entradas/saídas envolvida.

Nota didática

Overflow e ciclo-limite, ao contrário, são propriedades **sobre execuções**: perguntam se *alguma* sequência de entradas, ao longo de um número potencialmente ilimitado de amostras, algum dia causa overflow ou faz o sistema entrar em oscilação persistente. Isso exige desenrolar a equação de diferenças do filtro por um horizonte temporal que, em operação real, é ilimitado — exatamente o cenário em que BMC puro (desenrolamento até k) é estruturalmente incompleto: um resultado “sucesso” em profundidade k nunca descarta uma violação no passo $k + 1$. Daí a necessidade de uma técnica de indução (Artigo B) para elevar a garantia limitada a k passos para uma garantia válida *para todo tempo*.

2.4 (d) Aumentar `--x-size` não estabelece corretude

O parâmetro `-x-size` é o limite k de desenrolamento (“the verification bound, i.e., the number of times the digital system will be unwound”). Um resultado **successful** com um dado `-x-size=k` licencia apenas a afirmação **limitada**: “nenhuma violação da propriedade foi encontrada dentro das primeiras k iterações de operação do sistema digital” — o equivalente a $Base_k$ insatisfatível em BMC puro.

Resultado

Aumentar `-x-size` até a verificação passar **nunca** estabelece corretude do projeto *para todo tempo de operação*, porque overflow/ciclo-limite não têm um limiar de completude (*completeness threshold*) conhecido a priori: sempre é possível, em princípio, que a violação só se manifeste em $k+1$, $k+50$ ou qualquer profundidade maior que a testada. Um “successful” com `-x-size=k` prova estritamente que **não há** contraexemplo nos primeiros k passos — nunca que não existe contraexemplo algum. Só o passo indutivo do Artigo B (Exercício 5) fornece uma prova válida para *qualquer* número de iterações.

Exercício 3. Os três casos da k -indução (Artigo B)

Listing 1: Programa motivador, Artigo B (Fig. 2)

```

1 int main(void) {
2     uint64_t i = 1, sn = 0;
3     uint32_t n;
4     assume(n >= 1);
5     while (i <= n) {
6         sn = sn + a;
7         i++;
8     }
9     assert(sn == n * a);
10 }
```

3.1 (a) Por que BMC tradicional falha e quantos desdobramentos seriam necessários

`n` é `uint32_t` e não recebe cota superior além de `assume(n>=1)`, isto é, $n \in [1, 2^{32} - 1]$. BMC tradicional fixa um limite k de desenrolamento; para qualquer k escolhido, existe um valor de $n > k$ para o qual o laço ainda não terminou após k iterações — a *unwinding assertion* correspondente falharia, ou o BMC simplesmente nunca cobre esse caso. Como o próprio Artigo B afirma:

Citação direta

“BMC techniques find this difficult as the number of loop iterations, represented by `n`, is non-deterministically chosen; full unwinding would require $2^{32} - 1$ unfoldings.”
(Artigo B, Seção 3)

Resultado

São necessários $2^{32} - 1$ desdobramentos para uma verificação exaustiva — o número vem diretamente do maior valor representável por `uint32_t` ($2^{32} - 1$), já que o laço executa no máximo `n` vezes. Nenhum k fixo e finito, na prática, alcança esse limite em tempo/memória viáveis — daí a necessidade de k -indução.

3.2 (b) Transformação do caso base para $k = 2$

Citação direta

“[...] the base case simply inserts an unwinding assumption to the respective loop-free program P' , consisting of the termination condition σ after the loop, to ensure that it finds a counterexample of depth k without reporting any false incorrect result (i.e., to avoid unfolding loops partially).” (Artigo B, Seção 4.1.2)

Listing 2: Caso base, $k = 2$ (padrão da Fig. 9 do Artigo B)

```

1 int main(void) {
2     uint64_t i, sn = 0;
3     uint32_t n = nondet_uint();
4     assume(n >= 1);
5     i = 1;
6     if (i <= n) {                // copia 1
7         sn = sn + a;
8         i++;
9         if (i <= n) {            // copia 2
10            sn = sn + a;
11            i++;
12        }
13    }
14    assume(i > n);                // unwinding assumption
15    assert(sn == n * a);
16 }
```

Resultado

A instrução inserida ao final é um `assume(i > n)` (não um `assert`). Seu papel é **descartar** exatamente as execuções em que o laço **ainda não terminou** após as $k = 2$ cópias (isto é, os casos com $n > k$, onde `sn` só acumulou $k \cdot a$ em vez de $n \cdot a$). Sem esse `assume`, o `assert(sn==n*a)` falharia trivialmente para qualquer $n > k$ — não por um bug real, mas simplesmente porque o laço foi desenrolado poucas vezes demais. O `assume` é o que **impede que a ferramenta reporte um contraexemplo falso** (falso positivo de bug), causado por desenrolamento parcial, garantindo que o caso base só julgue execuções genuinamente completas ($n \leq k$).

3.3 (c) Transformação da condição de avanço para $k = 2$

Listing 3: Condição de avanço (*forward condition*), $k = 2$ (padrão da Fig. 10 do Artigo

B)

```

1  int main(void) {
2      uint64_t i, sn = 0;
3      uint32_t n = nondet_uint();
4      assume(n >= 1);
5      i = 1;
6      if (i <= n) {
7          sn = sn + a;
8          i++;
9          if (i <= n) {
10             sn = sn + a;
11             i++;
12         }
13     }
14     assert(i > n);           // check loop invariant
15     assert(sn == n * a);
16 }

```

Resultado

Não é a mesma instrução. O caso base insere um `assume(i>n)`; a condição de avanço insere um `assert(i>n)` — “*check loop invariant*”, nas palavras do próprio código-figura do artigo. A diferença de significado é o ato de fala: o **assume** do caso base *descarta* (filtra) as execuções não-concluídas para evitar falso positivo de bug; o **assert** da condição de avanço *prova ativamente* que o laço **sempre** termina dentro de k iterações (isto é, tenta demonstrar que $n \leq k$ é uma consequência necessária, o que só é possível se o laço for de fato limitado por k). Para este programa específico, como n é irrestrito em $[1, 2^{32} - 1]$, a condição de avanço **nunca** terá sucesso para nenhum k finito (sempre existe $n = k + 1$) — é precisamente por isso que a prova de corretude deste programa depende do **passo indutivo** (Exercício 4), não da condição de avanço.

3.4 (d) Por que a ordem base $\rightarrow$ avanço/indutivo importa

Citação direta

“The algorithm runs up to a maximum number of iterations and only increases the value of k if it can not falsify the property during the base case. Note that k is incremented only at the start of the **else** branch in line 9 of Fig. 3.” (Artigo B, Seção 4.1)

Pelo pseudocódigo da Fig. 3, a cada iteração do algoritmo: primeiro roda-se `base_case(P, phi, k)`; se ele encontra um contraexemplo, o algoritmo retorna **false imediatamente**; só no ramo `else` (caso base *não* achou bug) é que `k` é incrementado e então se tenta `forward_condition` e, falhando essa, `inductive_step`.

Nota didática

A ordem importa por **corretude**, não só por eficiência. A validade clássica da k -indução (Eq. 2 do Artigo B, Seção 2.3) exige a conjunção de **duas** condições: $Base_k$ insatisfável (nenhuma violação alcançável do estado inicial em até k passos) e $Step_k$ insatisfável (nenhuma transição de k estados bons para um estado ruim). O algoritmo da Fig. 3 obtém a metade “ $Base_k$ ” de forma *acumulada*: a cada iteração do laço externo, o caso base já foi checado (sem bug) para todos os valores de k percorridos até ali. Se a condição de avanço ou o passo indutivo fossem executados **primeiro**, o algoritmo poderia retornar **true** baseado apenas em $Step_k$ insatisfável, **sem** nunca ter confirmado $Base_k$ insatisfável no mesmo k — ou seja, poderia reportar corretude mesmo havendo um bug genuíno, alcançável do estado inicial em poucos passos, que simplesmente nunca chegou a ser checado. Checar o caso base primeiro não é apenas mais barato computacionalmente (o próprio artigo nota que rodar a condição de avanço ou o passo indutivo com k pequeno demais desperdiça recursos, já que “loops are usually unfolded at least two times”) — é o que garante que todo veredito **true** tem, de fato, tanto a âncora ($Base_k$) quanto o passo ($Step_k$) da indução matemática estabelecidos.

Exercício 4. O passo indutivo e suas transformações (Artigo B)

O laço `while(c) {E;}` é convertido em `A; while(c) { S; E; U; R; }` (Eq. 4).

4.1 (a) Função de cada bloco

Citação direta

“*A* is the code (or sequence of commands) responsible for assigning non-deterministic values to all loops variables, i.e., the state is havocked before the loop, *c* is the termination condition of the loop **while**, *S* is the code to store the current state of the program variables before executing the statements of *E*, *E* is the original body of the **while** loop, *U* is the code to update all state variables with local values after executing *E*, and *R* is the code to remove redundant states.” (Artigo B, Seção 4.1.2)

Bloco	Função
<i>A</i>	<i>Havoc</i> : atribui valores não determinísticos a todas as variáveis do laço, antes do laço começar — generaliza o estado inicial para que o model checker explore implicitamente todas as configurações possíveis.
<i>S</i>	Salva (armazena) o estado corrente das variáveis do programa antes de executar <i>E</i> na iteração atual — popula o vetor de estados sv .
<i>U</i>	Atualiza todas as variáveis de estado com os valores locais depois de executar <i>E</i> .
<i>R</i>	Remove estados redundantes (repetidos), via um assume que compara o vetor de estados salvo com o estado atual.

4.2 (b) O bloco *R* e a terminação do passo indutivo

O bloco *R* é um **assume** do tipo $sv_1[0] \neq cs_1 \wedge sv_1[0] \neq cs_1 \wedge sv_2[1] \neq cs_2 \wedge \dots$ (Eq. 8), isto é, exige que o estado da iteração atual seja **diferente** de todos os estados salvos nas iterações anteriores.

Nota didática

R elimina execuções em que a sequência de k estados percorridos **não é um caminho simples** (isto é, revisita um estado já visto — um ciclo dentro da janela de k iterações). Isso é necessário porque a validade clássica da k -indução sobre sistemas de transição (Sheeran et al. 2000; Eén & Sörensson 2003, citados no Artigo B) exige que os $k+1$ estados testados no passo indutivo sejam **genuinamente distintos**: sem essa restrição, o SMT solver poderia “provar” o passo indutivo trivialmente sobre um **ciclo curto** que se repete indefinidamente sem progresso real, produzindo uma conclusão *unsound* (o passo pareceria válido para qualquer n apenas porque o mesmo pequeno conjunto de estados foi revisitado, sem cobrir o comportamento real do programa para valores maiores). Ao proibir estados repetidos, R garante que, quando o passo indutivo termina com sucesso, a resposta é de fato útil (sound) — corresponde a uma cadeia real e crescente de estados do programa, não a um artefato de um laço de estados degenerado.

4.3 (c) Havoc apenas das variáveis do laço vs. Große et al. (todas as variáveis)**Citação direta**

“Similar to Donaldson et al. [8,9], we havoc only the variables that occur in the loop, i.e., all loops variables are assigned non-deterministic values. [...] Differently from our approach, Große et al. havoc all program variables, which makes it difficult to check for the reachability of an error since they do not provide enough information to constrain the havocked variables in the program.” (Artigo B, Seção 4)

Resultado

Ao randomizar **todas** as variáveis do programa (inclusive as que o laço nunca toca), o método de Große et al. destrói qualquer relação/invariante que essas variáveis não-relacionadas ao laço mantinham com o restante do estado — não sobra informação suficiente para **restringir** os valores havoceados de forma coerente com o que é de fato alcançável. Na prática, isso torna impossível raciocinar com precisão sobre se um estado de erro é **realmente** alcançável: o solver passa a considerar combinações de valores para variáveis não-relacionadas ao laço que nunca ocorreriam em uma execução real, inflando artificialmente o espaço de estados considerado (com risco de falsos alarmes) ou impedindo a prova de alcançabilidade (por faltar contexto). Havoeckear apenas as *variáveis do laço* (Definição 2/3 do Artigo B) preserva intactas as relações entre o restante do estado do programa e a lógica externa ao laço, mantendo o raciocínio sobre alcançabilidade preciso.

4.4 (d) Por que a hipótese $\{Post \wedge \neg(i \leq n)\}$ prova o caso geral**Citação direta**

“Lemma 1 If the induction hypothesis $\{Post \wedge \neg(i \leq n)\}$ holds for $k + 1$ consecutive iterations, then it also holds for k preceding iterations. [...] the SMT solver can easily verify Lemma 1 and conclude that `sn == n*a` is an inductive invariant relative to `n`.” (Artigo B, Seção 4.3.3)

A chave é que, no passo indutivo, o bloco A atribui valores **não-determinísticos genéricos** às variáveis do laço (via `nondet_uint()`) — o raciocínio não fixa nenhum valor concreto de `n`, nem assume que o estado inicial da janela de $k + 1$ iterações é alcançado a partir de um `n` específico. O que é provado é um **esquema geral**: “*para quaisquer $k+1$ estados consecutivos que satisfaçam $Post \wedge \neg(i \leq n)$, o mesmo vale para os k estados anteriores*”.

Resultado

Combinando essa hipótese genérica (válida para qualquer janela de $k+1$ iterações, não amarrada a um `n` concreto) com o caso base — que já estabeleceu `sn==n*a` para todo $n \leq k$ — obtém-se, por indução matemática clássica (base + passo), que a propriedade vale para **qualquer** $n \geq 1$, e não apenas até o limite k explorado explicitamente. Isso é exatamente o que separa k -indução de BMC puro: o passo indutivo, uma vez estabelecido, generaliza para infinitos valores de `n` sem precisar desenrolar mais nenhum deles.

Exercício 5. Integração entre os dois artigos

5.1 (a) Qual dos três casos fecha a lacuna de solidez do Artigo A

É o **passo indutivo**. O caso base apenas encontra bugs até profundidade k — não contribui nenhuma garantia positiva de ausência de erro além de k . A **condição de avanço** prova a propriedade “em todos os estados alcançáveis dentro de k iterações” — ainda uma garantia **limitada** a k , idêntica em espírito ao BMC puro que o Artigo A já descreve como insuficiente (“típicamente não sólida”). Apenas o **passo indutivo**, ao provar que a validade em k iterações consecutivas implica validade em $k+1$ (Lema 1), e combinado com o caso base como âncora, produz uma conclusão válida para **qualquer** número de iterações — a “técnica de indução” que o Artigo A antecipa como necessária para tornar overflow/ciclo-limite tão sólidos quanto estabilidade/fase mínima.

Nota didática

Os outros dois casos são dispensáveis *para esse fim específico*? A condição de avanço é dispensável para fechar a lacuna de solidez em sistemas genuinamente de operação ilimitada (ela só ajuda quando o laço de fato termina dentro de algum k pequeno, um atalho de eficiência, não uma fonte de solidez *unbounded*). Já o **caso base não é dispensável**, mesmo para esse fim específico: pela Eq. 2 do Artigo B, a validade da k -indução exige $Base_k$ insatisfável **junto com** $Step_k$ insatisfável — sem confirmar o caso base, o passo indutivo sozinho não garante ausência de bug nos primeiros k passos (ver Exercício 3d). O caso base é a âncora lógica sem a qual o passo indutivo não licencia nenhuma conclusão sobre o programa completo.

5.2 (b) Alterações necessárias no fluxo do DSVerifier

Pela Fig. 1 do Artigo A, a k -indução do Artigo B é implementada como transformações aplicadas **no nível da representação intermediária GOTO-program**, durante a conversão $C/C++ \rightarrow GOTO$ (“these transformations take place at the intermediate representation level, during the conversion of the C/C++ program into a GOTO-program”), ou seja, **dentro** do próprio ESBMC, entre os estágios “C Parser” e “GOTO Symex”.

Resultado

O módulo DSVerifier em si (Initialization $\rightarrow$ Validation $\rightarrow$ Instrumentation, antes do C Parser) **não precisa de nenhuma alteração**: o arquivo ANSI-C que ele gera continua idêntico. A mudança ocorre exclusivamente na **etapa 5** da metodologia da Fig. 2 do Artigo A (“Verify Using a BMC tool”): em vez de invocar o ESBMC em modo de BMC puro (desenrolamento simples até `-x-size`), invoca-se o ESBMC com a k -indução habilitada (na prática, as flags `-k-induction -k-step <k>` usadas nos experimentos do Artigo B). Como as transformações de k -indução agem automaticamente sobre *qualquer* laço presente no GOTO-program, elas se aplicam também aos laços de verificação que a própria instrumentação do DSVerifier injeta, sem que o DSVerifier precise “saber” que a k -indução existe.

5.3 (c) O código gerado pelo DSVerifier respeita as restrições do passo indutivo?

Citação direta

“[...] the inductive step of our k -induction algorithm does not support heap-manipulating programs and unbounded recursion [...] Most of the incorrect results provided by ESBMC are related to bugs in the coding of the inductive step transformations, because we do not support `continue` and `return` (from inside a loop) statements.” (Artigo B, Introdução e Seção 5.2)

Resultado

Sim, respeita. As realizações de filtros/controladores digitais que o DSVerifier gera (formas diretas, formas delta e cascata — vetores de coeficientes `ds.a/ds.b` de tamanho fixo conhecido em tempo de geração, via `a_size/b_size`) são núcleos numéricos **estáticos, simples e não-recursivos**: um laço (ou sequência de laços) que acumula somas e produtos sobre *arrays* de tamanho fixo, sem alocação dinâmica (*heap*), sem chamadas recursivas (a realização de um filtro é inerentemente iterativa, espelhando diretamente a equação de diferenças) e sem desvios de controle antecipados (`continue/return`) dentro do laço de processamento de amostras.

A característica decisiva é que código de realização de sistemas digitais é um **“laço de aritmética em linha reta”**: em cada iteração executa-se sempre a mesma sequência fixa de multiplicações e somas sobre estados/coeficientes de tamanho estático, sem ramificação de controle que dependa de dados nem estrutura de dados dinâmica — exatamente a classe de programas para a qual as transformações do Artigo B (havoc de variáveis esca-

lares/vetoriais de tamanho fixo conhecido, bloco R sobre um `statet` também de tamanho fixo) foram desenhadas.

5.4 (d) Mais resultados corretos e mais *wrong proofs*: o que pesa mais?

Citação direta

“Wrong proofs is the number of false positive results (i.e., the tool reports SAFE incorrectly).” (Artigo B, Seção 5.2) — Tabela 1: ESBMC paralelo, 3153 resultados corretos / 265 *wrong proofs*; CPAchecker, 2349 resultados corretos / 26 *wrong proofs*.

	ESBMC sequencial	ESBMC paralelo	CPAchecker
Resultados corretos	3071	3153	2349
<i>Wrong proofs</i> (falso SAFE)	111	265	26

Resultado

Para um engenheiro decidindo se adota a ferramenta na verificação de um controlador embarcado **crítico**, o número de *wrong proofs* pesa muito mais do que o número bruto de resultados corretos — e a razão é a assimetria de consequências entre os dois tipos de erro:

- Um **timeout/unknown** (nenhum veredito) é um modo de falha *seguro*: a engenharia simplesmente não ganha um certificado formal e recorre a outra técnica (teste, revisão manual, outra ferramenta) — nenhum dano é feito, nenhuma confiança falsa é criada.
- Uma **wrong proof** é um modo de falha *perigoso*: a ferramenta certifica como SAFE um controlador que na verdade contém um defeito. Isso gera **confiança falsa** — exatamente o cenário que a verificação formal existe para eliminar. Um controlador defeituoso pode ser embarcado em produção *acreditando-se* que foi formalmente verificado.

Relativamente, o ESBMC paralelo erra com falso-SAFE em $265/3153 \approx 8,4\%$ dos casos resolvidos, contra $26/2349 \approx 1,1\%$ do CPAchecker — quase $8\times$ mais frequente. Vale notar ainda que a própria implementação paralela do ESBMC **piora** esse número em relação à sequencial ($111 \rightarrow 265$ *wrong proofs*), precisamente porque processos que estourariam memória/tempo na versão sequencial (e ficariam honestamente “unknown”) são convertidos em vereditos definitivos na versão paralela — um ganho de cobertura obtido à custa de solidez. Para certificação de um sistema crítico, a escolha razoável é privilegiar a menor taxa de falso-SAFE (CPAchecker, ou ESBMC sequencial em vez do paralelo), mesmo abrindo mão de cobertura bruta — e, idealmente, usar múltiplas ferramentas/técnicas em conjunto, já que nenhuma delas tem taxa zero de *wrong proofs*.

Referências

- [1] H. I. Ismail, I. V. Bessa, L. C. Cordeiro, E. B. de Lima Filho, J. E. Chaves Filho. *DSVerifier: A Bounded Model Checking Tool for Digital Systems*. SPIN 2015, LNCS 9232, pp. 126–131.
- [2] M. Y. R. Gadelha, H. I. Ismail, L. C. Cordeiro. *Handling Loops in Bounded Model Checking of C Programs via k -Induction*. STTT, 2017.